

From the File Kafka Lab Use Cases, write in the proper format which we have to divided into the 2 groups and each group has few terminal also, whichever we are change from system to system tell that me "<<>>" write that inside it s

Here is the step-by-step breakdown of the Kafka Lab commands from the provided source, divided into **Group A (Part A)** and **Group B (Part B)**.

TXT

All system-dependent parameters (such as path locations, usernames, cluster IDs, topic names, file paths, and host IP addresses/ports) that need to be modified based on your system or use case are enclosed in << >>.

TXT

Group A: Kafka Core Commands (Part A)

Terminal 1: Environment Setup & Kafka Broker Server

1. Download and Extract Kafka Archive:

Bash



```
cd ~  
wget -c https://repo.huaweicloud.com/repository/apache/kafka/<<3.7.2>>/kafka_<<2.13-3.7.2>>.tgz[cite: 1]  
tar -xzf kafka_<<2.13-3.7.2>>.tgz[cite: 1]  
cd ~/kafka_<<2.13-3.7.2>>[cite: 1]
```

2. Verify Java Environment:

Bash



```
java -version[cite: 1]  
export JAVA_HOME=/usr/lib/jvm/<<java-8-openjdk-amd64>>[cite: 1]  
export PATH=$JAVA_HOME/bin:$PATH[cite: 1]
```

3. Generate Cluster ID and Format Storage (KRaft Mode):

Bash



```
KAFKA_CLUSTER_ID="$(bin/kafka-storage.sh random-uuid)"[cite: 1]  
echo $KAFKA_CLUSTER_ID[cite: 1]
```

```
bin/kafka-storage.sh format -t "<<$KAFKA_CLUSTER_ID>>" -c config/kraft/server.pr
```

4. Start Kafka Broker Server:

Bash



```
bin/kafka-server-start.sh config/kraft/server.properties
```

Terminal 2: Topic Administration

1. Create Topic:

Bash



```
cd ~/kafka_<<2.13-3.7.2>>[cite: 1]  
bin/kafka-topics.sh --create --topic <<payments>> --bootstrap-server <<localhost
```

2. Verify / Describe Topic:

Bash



```
bin/kafka-topics.sh --describe --topic <<payments>> --bootstrap-server <<localho
```

Terminal 3: Console Producer

1. Start Kafka Console Producer:

Bash



```
cd ~/kafka_<<2.13-3.7.2>>[cite: 1]  
bin/kafka-console-producer.sh --topic <<payments>> --bootstrap-server <<localhost>>
```

2. Produce Sample Data Records:

Plaintext



```
<<acc101>>|{"txn_id":1,"amount":2500,"type":"DEBIT"}[cite: 1]  
<<acc102>>|{"txn_id":2,"amount":99000,"type":"CREDIT"}[cite: 1]  
<<acc101>>|{"txn_id":3,"amount":120,"type":"DEBIT"}[cite: 1]  
<<acc103>>|{"txn_id":4,"amount":45000,"type":"DEBIT"}[cite: 1]  
<<acc101>>|{"txn_id":5,"amount":800,"type":"DEBIT"}[cite: 1]
```

Terminal 4: Console Consumer

1. Start Kafka Console Consumer:

Bash



```
cd ~/kafka_<<2.13-3.7.2>>[cite: 1]  
bin/kafka-console-consumer.sh --topic <<payments>> --bootstrap-server <<localhost>>
```

Group B: Kafka Connect Pipeline (Part B)

[cite: 1] TXT

Terminal 1: Connector Configuration & Properties File Setup

1. Create Directories and Sample Log File:

Bash



```
cd ~/kafka_<<2.13-3.7.2>>[cite: 1]  
mkdir -p ~/<<connect-demo>>/source mkdir -p ~/<<connect-demo>>/sink[cite: 1]
```

```
nano ~/<<connect-demo>>/source/<<transactions.log>>[cite: 1]
```

Add sample data to `transactions.log`:

JSON



```
{ "txn_id":101,"account":"acc201","amount":1500,"city":"Bengaluru"}[cite: 1]
{ "txn_id":102,"account":"acc202","amount":32000,"city":"Mumbai"}[cite: 1]
{ "txn_id":103,"account":"acc203","amount":700,"city":"Delhi"}[cite: 1]
```

2. Configure Standalone Properties:

Bash



```
nano ~/kafka_<<2.13-3.7.2>>/config/connect-standalone.properties[cite: 1]
```

Append to the bottom of the file:

Properties



```
plugin.path=/home/<<ubuntu>>/kafka_<<2.13-3.7.2>>/libs[cite: 1]
```

3. Configure File Source Properties (`file-source.properties`):

Bash



```
nano ~/<<connect-demo>>/file-source.properties[cite: 1]
```

Content:

Properties



```
name=<<txn-file-source>>[cite: 1]
connector.class=FileStreamSource[cite: 1]
tasks.max=1[cite: 1]
file=/home/<<ubuntu>>/<<connect-demo>>/source/<<transactions.log>>[cite: 1]
```

```
topic=<<bank-transactions>>[cite: 1]
key.converter=org.apache.kafka.connect.storage.StringConverter[cite: 1]
value.converter=org.apache.kafka.connect.storage.StringConverter[cite: 1]
```

4. Configure File Sink Properties (`file-sink.properties`):

Bash



```
nano ~/<<connect-demo>>/file-sink.properties[cite: 1]
```

Content:

Properties



```
name=<<txn-file-sink>>[cite: 1]
connector.class=FileStreamSink[cite: 1]
tasks.max=1[cite: 1]
file=/home/<<ubuntu>>/<<connect-demo>>/sink/<<transactions_archive.txt>>[cite: 1]
topics=<<bank-transactions>>[cite: 1]
key.converter=org.apache.kafka.connect.storage.StringConverter[cite: 1]
value.converter=org.apache.kafka.connect.storage.StringConverter[cite: 1]
```

Terminal 2: Connect Topic Creation

1. Create and Verify Connect Kafka Topic:

Bash



```
cd ~/kafka_<<2.13-3.7.2>>[cite: 1]
bin/kafka-topics.sh --create --topic <<bank-transactions>> --bootstrap-server <<
bin/kafka-topics.sh --describe --topic <<bank-transactions>> --bootstrap-server
```

Terminal 5: Run Kafka Connect Standalone Worker

1. Start Kafka Connect Process:

Bash



```
cd ~/kafka_<<2.13-3.7.2>>[cite: 1]  
bin/connect-standalone.sh config/connect-standalone.properties ~/<<connect-demo>
```

Terminal 6: Verification & Data Pipeline Testing

1. Verify REST API Connectors Status:

Bash



```
curl -s <<localhost:8083>>/connectors[cite: 1]  
curl -s <<localhost:8083>>/connectors/<<txn-file-source>>/status[cite: 1]
```

2. Verify Topic Consumption:

Bash



```
cd ~/kafka_<<2.13-3.7.2>>[cite: 1]  
bin/kafka-console-consumer.sh --topic <<bank-transactions>> --bootstrap-server <
```

3. Verify Sink Archive File Output:

Bash



```
cat ~/<<connect-demo>>/sink/<<transactions_archive.txt>>[cite: 1]
```

4. Append New Records to Source (Test Dynamic Streaming):

Bash



```
echo '{"txn_id":104,"account":"acc204","amount":88000,"city":"Chennai"}' >> ~/<<  
echo '{"txn_id":105,"account":"acc205","amount":260,"city":"Hyderabad"}' >> ~/<<
```

5. Verify Updated Output in Sink File:

